## Clustering: grouping without labels, where similarity is a design choice

Having covered how to fit models when labels are available, the lecture switches to **clustering** — a classic *unsupervised* problem in which we decide on the "similarity" of examples in order to separate them into distinct, "natural" groups. The repeated scare quotes around "natural" are the central caveat: the entire outcome hinges on what we mean by *similar*, and similarity is operationalized as a **distance measure**. Whatever measure we pick determines what our groups look like — change the metric, change the clusters.

The setup assumes we know there are $k$ different groups in the training data but do not know the labels; the running example uses $k = 2$. In the standard formalism (from the k-means literature), we partition $n$ observations into $k$ clusters such that each observation belongs to the cluster with the **nearest mean** (the cluster center or *centroid*). Geometrically, this carves the data space into **Voronoi cells**, one per centroid. Historically the method grew out of **vector quantization** in signal processing, which is why it is framed as a way of compressing many observations into a few representative centers.

## The algorithm: random exemplars, assign, update, repeat

The recipe given in lecture:

1. Pick $k$ samples — *at random*, with a pointed question mark attached — to serve as **exemplars**: prototype members of each group.
2. Cluster the remaining samples by minimizing the distance between samples in the same cluster (this is the **objective function**). Operationally, this means assigning each sample to the group whose exemplar is closest.
3. Find the median example in each cluster and promote it to be the new exemplar.
4. Repeat until no change.

This is an objective function being improved iteratively — *assign, update, assign, update* — until things settle down. The Wikipedia material identifies this exact structure: the most common algorithm uses an **iterative refinement** technique that alternates between two steps starting from an initial set of $k$ means $m_1^{(1)}, \dots, m_k^{(1)}$. It is known as **Lloyd's algorithm** (or "naïve k-means," since faster alternatives exist).

Two properties matter for understanding *why* the recipe works and where it can fail:

- **It finds a local optimum, not the global one.** The underlying optimization problem is computationally difficult (**NP-hard**), but the heuristic converges quickly to a local optimum. This is precisely why the random initialization carries a question mark: different starting exemplars can lead to different final clusterings.
- **Mean vs. median is not cosmetic.** K-means minimizes within-cluster *variances* (squared Euclidean distances), not plain Euclidean distances — minimizing the latter would be the harder **Weber problem**. The mean optimizes squared errors, whereas only the geometric median minimizes true Euclidean distance; hence variants like **k-medians** and **k-medoids** can find better Euclidean solutions. The lecture's "promote the median example" update step sits squarely in this median-based family.

The same assign/update skeleton also appears in the **expectation–maximization algorithm for Gaussian mixtures**: both use cluster centers to model the data and refine them iteratively. The difference is expressive power — k-means tends to find clusters of comparable spatial extent, while a Gaussian mixture model allows clusters of different shapes.

## The objective function: within-cluster sum of squares

Formally, given observations $(x_1, \dots, x_n)$, each a $d$-dimensional real vector, k-means seeks a partition $S = \{S_1, \dots, S_k\}$ (with $k \leq n$) minimizing the **within-cluster sum of squares (WCSS)**:

$$\arg\min_S \sum_{i=1}^{k} \sum_{x \in S_i} \|x - \mu_i\|^2 \;=\; \arg\min_S \sum_{i=1}^{k} |S_i| \operatorname{Var} S_i$$

where $\mu_i$ is the **centroid** of cluster $S_i$,

$$\mu_i = \frac{1}{|S_i|} \sum_{x \in S_i} x,$$

and $\|\cdot\|$ is the usual $L_2$ norm. Three equivalences explain why this objective behaves well:

- Minimizing WCSS is the same as minimizing the **pairwise squared deviations** of points sharing a cluster,

$$\arg\min_S \sum_{i=1}^{k} \frac{1}{|S_i|} \sum_{x,y \in S_i} \|x - y\|^2,$$

  via the identity $|S_i| \sum_{x \in S_i} \|x - \mu_i\|^2 = \frac{1}{2} \sum_{x,y \in S_i} \|x - y\|^2$. Compactness around a center and mutual closeness of members are the same goal.
- Because total variance is fixed, minimizing WCSS is equivalent to **maximizing the between-cluster sum of squares (BCSS)** — pushing clusters apart is the flip side of pulling members together. This deterministic relationship connects to the **law of total variance** in probability theory.
- After each iteration the WCSS **monotonically decreases**, yielding a nonnegative monotonically decreasing sequence. That monotone guarantee is what makes "repeat until no change" a legitimate termination rule: the algorithm cannot oscillate, and must converge (to a local optimum).

## The distance measure decides the answer: one attribute versus two

A concrete dataset makes the "similarity is a choice" point vivid: **height** on the horizontal axis (running 60 to 90) versus **weight** on the vertical axis (roughly 200 to 350), with $k = 2$.

- **Similarity based on weight alone.** The natural two-group split is a *horizontal* line, drawn at a weight of roughly 240: everything above is "heavy," everything below is "light." This yields one legal partition — a large group in the 260-to-335 weight range and three examples down around 190 to 210 — but it **throws away the height information entirely**.
- **Similarity based on height alone.** Now the dividing line is *vertical*, at a height of about 74–75. The consequences are visibly worse: the example sitting at height 73 but weight 310 — short yet very heavy — gets lumped in with the three light examples on the left, while the two examples at height 78 and weight 265–275 join the tall, heavy group. Height alone produces a rather different partition than weight alone, and intuitively a poorer one.
- **Similarity using both attributes.** Measuring distance in the full two-dimensional height–weight space — for instance with ordinary **Euclidean distance** — gives the clusters your eye actually draws: a red dashed circle around the tall, heavy group in the upper right and a green dashed circle around the short, light group in the lower left, separated by a blue dashed line. Two clean, compact, well-separated clusters — what the iterative algorithm with a sensible distance over both features should converge to.

The lesson generalizes: neither single attribute, used by itself, captures what we mean by "similar." The geometry of the chosen feature space *is* the notion of similarity, and the resulting Voronoi-style partition inherits all of its blind spots.

## Real structure versus labeled structure: when the best clusters are wrong

Now the crucial twist: **suppose the data was labeled**. Coloring each example by its true label — blue for one class, red for the other — reveals that the big upper-right group is blue *except for two points*: the examples at height 78 with weights 265 and 275, exactly the ones the geometric clustering confidently placed in the upper cluster. They are red — the same class as the short, light examples in the lower left.

So the "best" clusters, those minimizing within-cluster distance using both attributes, **do not match the true labels**. The implication is structural, not incidental:

- Structure found in an unsupervised way need not correspond to the distinction we actually care about.

- The label may depend on something not captured by height and weight at all, or on a more complicated combination of the features than plain distance allows.
- Clustering did find *real* structure in the data — it simply wasn't the *labeled* structure.

This limitation is exactly what motivates turning the problem around: rather than hoping the natural groups align with the labels, use the labels directly to learn a classifier — the supervised approach taken up next.

## Origins, relatives, and known failure modes of k-means

The historical record explains both the method's shape and its documented weaknesses:

- **Origins.** The term "k-means" was first used by James MacQueen in 1967, though the idea goes back to Hugo Steinhaus in 1956. The standard algorithm was proposed by Stuart Lloyd at Bell Labs in 1957 as a technique for **pulse-code modulation** — representing analog signals with a limited set of discrete values, using clustering to reduce data volume while preserving signal quality — though it was not published in a journal until 1982. Edward Forgy published essentially the same method in 1965, hence the name **Lloyd–Forgy algorithm**. Early applications were in signal processing and data compression (vector quantization); as computing power grew, the method spread to pattern recognition, statistical classification, and early machine learning on large datasets, prized for its simplicity and computational efficiency.
- **Recognized limitations.** Sensitivity to **initial centroid placement** — the source of the lecture's question mark over random exemplar selection — and difficulty handling **non-spherical clusters** were noted early on, motivating improved initialization techniques and clustering methods.
- **Extensions. Fuzzy c-means** lets data points belong to multiple clusters with varying degrees of membership; **kernel k-means** uses kernel functions to identify non-linearly separable clusters — directly addressing the case where the label depends on a combination of features that plain Euclidean distance cannot see.
- **A namesake trap.** K-means has only a loose relationship to the **k-nearest neighbor classifier**, a *supervised* technique with which it is often confused because of the name. There is a genuine bridge, though: applying the 1-nearest-neighbor classifier to the cluster centers obtained by k-means classifies new data into the existing clusters — the **nearest centroid classifier**, also known as the **Rocchio algorithm**.

## Classification with labeled data: turning a cloud of points into a decision rule

The height-versus-weight scatter returns, but with a crucial change: every point now carries a label. The axes span heights of roughly 60–90 horizontally and weights of about 200–350 vertically, and the labeled groups occupy visibly different regions. The blue points cluster in the upper portion of the plot, with weights around 300–335 at heights in the mid-70s to 80; the red points sit lower, with weights from about 190 up to around 275; and a couple of black points appear at weights of 200–250 near heights of 70–72.

What the labels buy is a *learned model*: a nearly horizontal blue line at a weight of about 290, running across the entire plot. Every blue point falls above it; every red and black point falls below it. The line therefore acts as a classifier — everything above is assigned one class, everything below the other. The payoff is that a brand-new, unlabeled example requires no further knowledge: given its height and weight, we simply ask which side of the line it lands on. The labels converted pure geometry into semantics — position in feature space now means class membership.

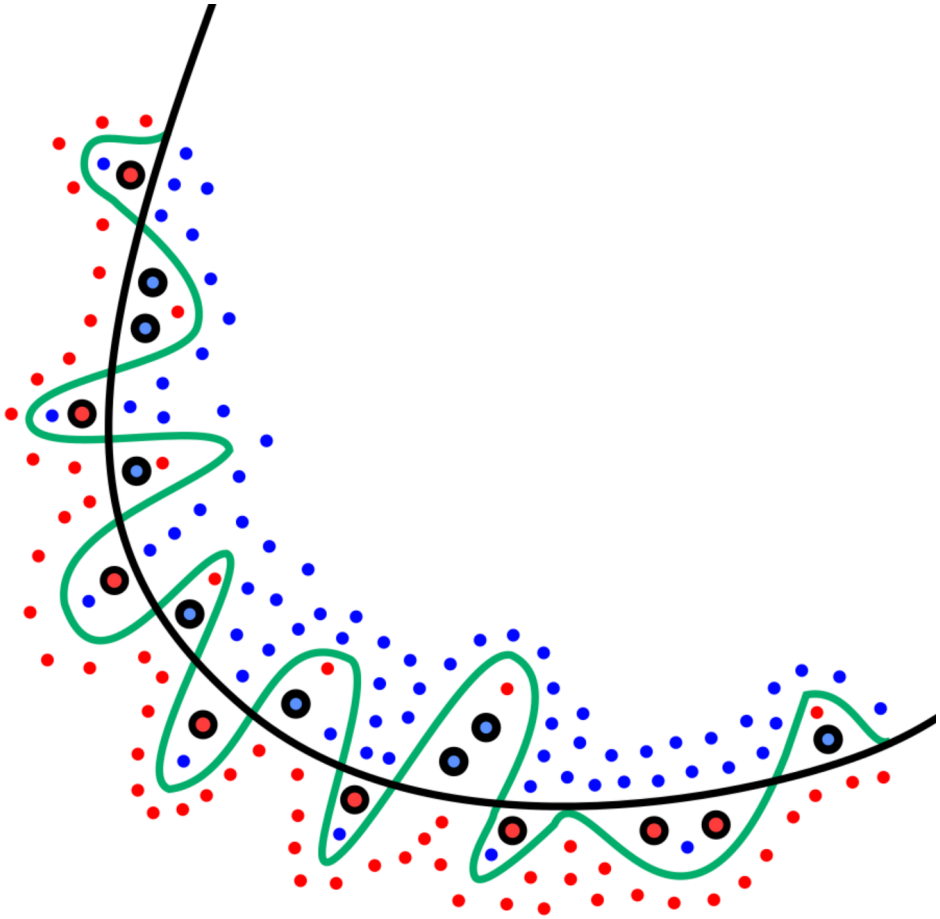
## Two families of learning methods — and the price of perfect separation

This example motivates the two broad flavors of machine learning method:

1. **Learning from unlabeled data** by clustering the training data into groups of nearby points — the situation when the height–weight data had no labels. The resulting clusters can then assign labels to new data based on which group they fall nearest.

2. **Learning from labeled data** by finding models that separate labeled groups of similar data from other groups — exactly what the horizontal line at weight  290 does.

An important caveat attaches to the second flavor: it may not be possible to separate the groups perfectly without **overfitting**. If you insist on a separator that gets every single training example right, you may be fitting quirks of this particular dataset rather than the underlying structure. The Wikipedia account makes the mechanism precise: overfitting is "the production of an analysis that corresponds too closely or exactly to a particular set of data and may therefore fail to fit to additional data or predict future observations reliably." An overfitted model contains more parameters than can be justified by the data, and its essence is *unknowingly extracting some of the residual variation — i.e., noise — as if that variation represented the underlying model structure.* The extreme case is stark: when the number of parameters equals or exceeds the number of observations ($p \geq n$), a model can achieve perfect predictions on the training data purely by memorizing it — in linear regression, $p$ variables with $p$ data points let the fitted line pass exactly through every point — and such a model typically fails severely when making real predictions.



*Source: Wikipedia, article "Overfitting".*

The figure captures the failure mode: the green curve tracks the training data almost perfectly, yet because it is too dependent on that data it incurs a higher error rate on new unseen data (the black-outlined dots) than the smoother black model. Even honest models degrade somewhat out of sample — a phenomenon called *shrinkage*, where the fitted relationship performs less well on new data and measures like the coefficient of determination shrink relative to the original fit. The opposite pole is **underfitting**: a model missing

parameters or terms that a correctly specified model would have, such as fitting a linear model to nonlinear data, which yields poor predictive performance. The goal, as Burnham & Anderson put it, is "properly balancing the errors of underfitting and overfitting"; in regression terms, the mean squared error decomposes into random noise, approximation bias, and variance in the estimated function, and the bias–variance tradeoff is the standard tool for navigating this. Overfitting is, in Occam's-razor terms, the use of more adjustable parameters — or a more complicated approach — than is ultimately optimal.

The practical remedy the lecture emphasizes is that classification decisions need not demand perfection: we can trade off **false positives versus false negatives**, placing the boundary wherever the relative cost of the two mistake types dictates. Either way — clustered or separated — the resulting models can assign labels to new data, which is the entire point.

## The ingredients every machine-learning method shares

Before details, it pays to see that all machine learning methods require the same set of ingredients:

1. Choosing the **training data** and an **evaluation method**;
2. A **representation of the features**;
3. A **distance metric** for feature vectors;
4. An **objective function** and constraints;
5. An **optimization method** for actually learning the model.

Ingredients 2 and 3 — representation and distance — are the focus of this lecture; the objective function and the optimization procedure come later. The division is principled: representational choices determine *what comparisons are even possible* between examples, before any optimizer ever runs. A poor choice of features or metric cannot be repaired downstream by clever optimization.

## "All models are wrong, but some are useful": features are always lossy

The first lesson of feature representation is that **features never fully describe the situation**. The guiding aphorism comes from the British statistician George E. P. Box, who used the phrase in a 1976 paper to argue that while no model is ever completely accurate, simpler models can still provide valuable insights if applied judiciously. The longer form appears in Box and Draper's 1987 book, in a section on approximating functions: "The fact that the polynomial is an approximation does not necessarily detract from its usefulness because all models are approximations. Essentially, all models are wrong, but some are useful." Later commentators reinforced the point from different angles: McCullagh and Nelder (1983) observed that modeling is a creative process in which some models are better than others even though none can claim eternal truth; Burnham and Anderson (2002) noted that models, being simplifications of reality, vary in usefulness from highly useful to essentially useless; and David Hand (2014) reiterated that models exist to aid understanding or decision-making about the real world.

Applied to machine learning, this means our feature vectors are always a *partial, lossy description of reality.* The art of feature engineering is choosing features that are wrong in ways that don't matter — simplifications that discard irrelevancies while preserving whatever drives the label.

## Feature engineering as maximizing the signal-to-noise ratio

**Feature engineering** is the task of representing examples by feature vectors that will facilitate generalization. A concrete scenario makes the stakes clear: suppose you want to use 100 examples from past years to predict, at the start of the term, which students will get an A. Some candidate features are surely helpful — GPA, prior programming experience — though notice that neither is a perfect predictor. Others are actively harmful: birth month, eye color. Including them invites the model to latch onto accidental patterns in those particular 100 examples that have nothing to do with who does well, and the resulting model will not generalize to new students. This is precisely the overfitting mechanism described earlier, and statistics has a name for how it arises with many irrelevant variables: **Freedman's paradox** — with a large set of

explanatory variables that actually have no relation to the outcome being predicted, some will in general be falsely found to be statistically significant and retained, thereby overfitting the model.

The design goal is to maximize the ratio of useful input to irrelevant input — what engineers call the **Signal-to-Noise Ratio (SNR)**. Formally, SNR compares the level of a desired signal to the level of background noise, defined as the ratio of signal power to noise power, often expressed in decibels; a ratio higher than 1:1 (greater than 0 dB) indicates more signal than noise. For a random signal $S$ measured against random noise $N$,

$$\mathrm{SNR} = \frac{\mathrm{E}[S^2]}{\mathrm{E}[N^2]},$$

where E denotes expected value. If the signal is simply a constant value $s$, this simplifies to SNR $= s^2/\mathrm{E}[N^2]$, and when the noise has zero mean — the common case — the denominator is its variance $\sigma_N^2$. The interpretation carries over directly: a high SNR means the signal is clear and easy to detect or interpret, while a low SNR means the signal is corrupted or obscured by noise and difficult to distinguish or recover. Mapped onto feature selection: **good features raise the signal; bad features just add noise.**

## A one-example training set: why a single cobra cannot generalize

These ideas come to life in a small running example. The training table has five features — egg-laying, scales, poisonous, cold-blooded, and number of legs — and one label: *reptile*. So far, the entire training set consists of a single row:

| Egg-laying | Scales | Poisonous | Cold-blooded | Legs | Reptile |
|---|---|---|---|---|---|
| true | true | true | true | 0 | yes |

The initial model is stated plainly: **not enough information to generalize**. With a single example, the best any learning algorithm can do is memorize it. Is being poisonous the key to being a reptile? Having zero legs? Being cold-blooded? There is no way to tell which of these features matter and which are incidental, because there is nothing to compare against. One data point against five features is a terrible signal-to-noise ratio — every feature is perfectly "predictive" of the single observed label, which is exactly the $p \geq n$ memorization regime in which overfitting is guaranteed. Clearly more examples are needed; as they are added, the model is forced to start making decisions about which features actually carry the signal.

## From animals to feature vectors

To reason about similarity computationally, each animal is encoded as a vector of numeric features. In the running example, every animal gets five coordinates: the first four are binary attributes (1 = present, 0 = absent), and the fifth counts legs, a value anywhere from 0 to 4. The initial three animals are:

- Rattlesnake: $[1, 1, 1, 1, 0]$
- Boa constrictor: $[0, 1, 0, 1, 0]$
- Dart frog: $[1, 0, 1, 0, 4]$

Both snakes score 0 on the legs coordinate; the dart frog scores 4. Once animals are points in a five-dimensional space, the question "how alike are these animals?" becomes "how far apart are these points?" — and any distance function defined on vectors can answer it.

## Euclidean distance: the measuring stick

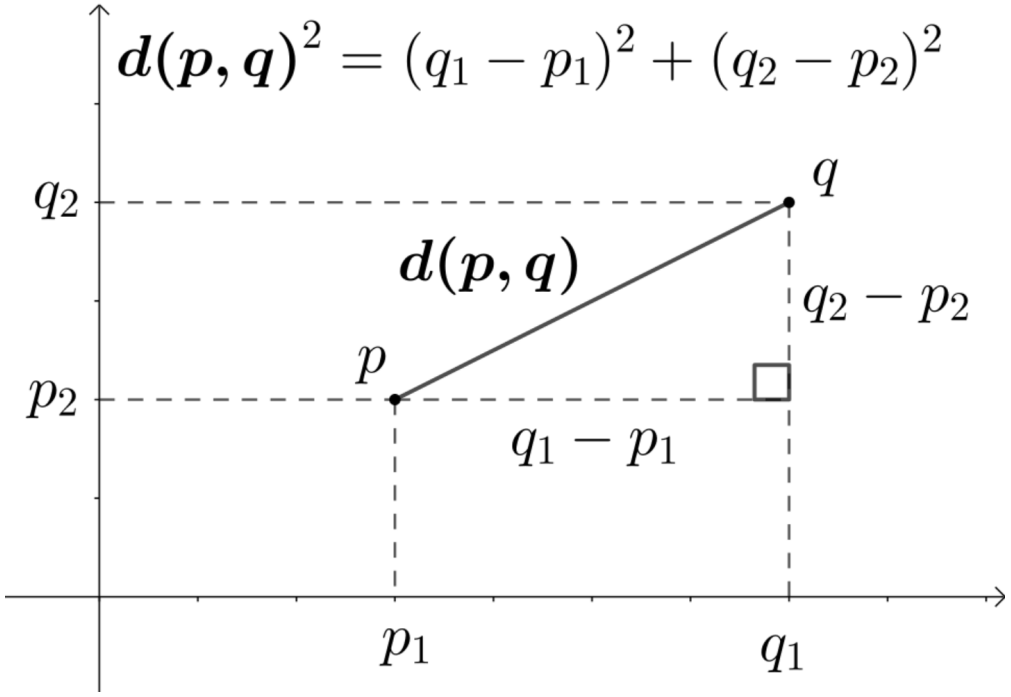
The natural choice is **Euclidean distance**: in mathematics, this is the length of the straight-line segment between two points in a Euclidean space. Because it is calculated from Cartesian coordinates using the

Pythagorean theorem, it is occasionally called the *Pythagorean distance*. On the real line, the distance between points $p$ and $q$ is simply their absolute difference,

$$d(p, q) = |p - q|,$$

or equivalently $d(p, q) = \sqrt{(p - q)^2}$ — a more complicated-looking form that leaves positive numbers unchanged but generalizes readily to higher dimensions. In the Euclidean plane, for $p = (p_1, p_2)$ and $q = (q_1, q_2)$, the Pythagorean theorem applied to a right triangle whose hypotenuse joins the two points gives

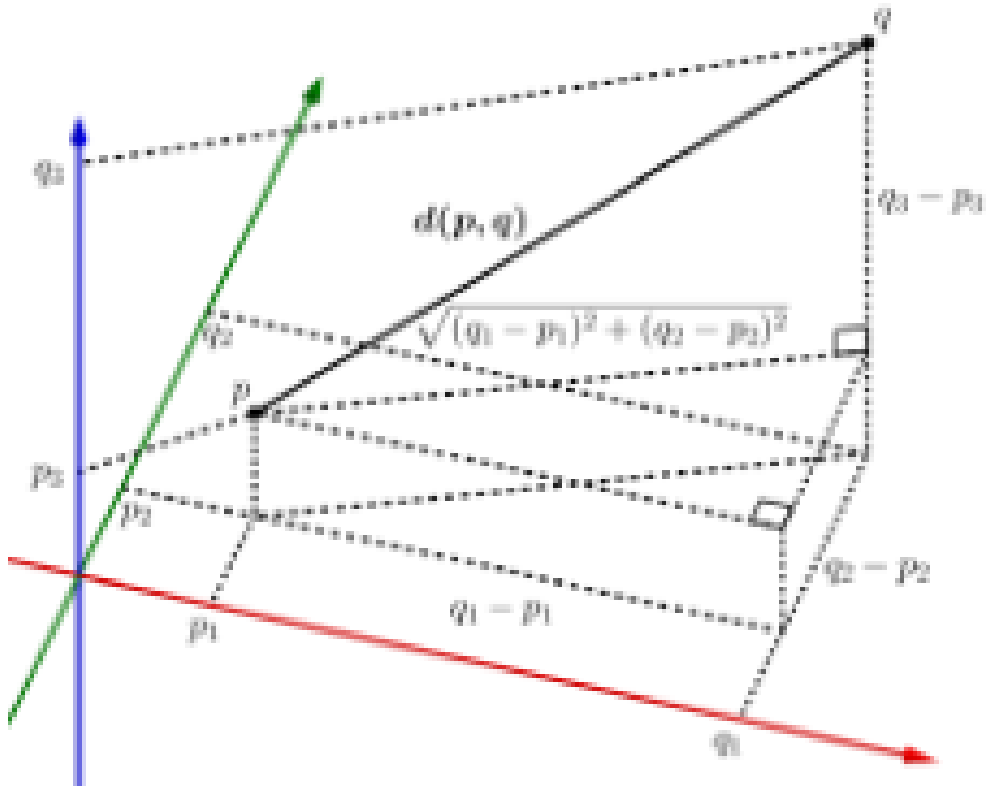
$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2},$$




*Source: Wikipedia, article "Euclidean distance".*

and repeated application of the same theorem yields the general formula for points given by Cartesian coordinates in $n$-dimensional space:

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \cdots + (p_n - q_n)^2}.$$




*Source: Wikipedia, article "Euclidean distance".*

This is exactly what the animal example needs: each animal is a point in $\mathbb{R}^5$, and its distance to another animal is the Euclidean norm of the vector difference, written compactly as $d(p, q) = \|p - q\|$. Euclidean distance is the prototypical example of a distance in a metric space — it obeys all the defining properties of a metric, and even stronger structure such as Ptolemy's inequality, $d(p, q) \cdot d(r, s) + d(q, r) \cdot d(p, s) \geq d(p, r) \cdot d(q, s)$. One detail matters greatly for what follows: in many statistics and optimization applications, the *square* of the Euclidean distance is used instead of the distance itself. Squaring is what allows a single large coordinate gap to dominate the total.

## A distance matrix that matches intuition

Applying the formula to all pairs of the original three animals produces a distance matrix:

- **Rattlesnake boa constrictor:** they differ in exactly two of the binary features, so $d = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.414$.
- **Rattlesnake dart frog:** $d = \sqrt{0 + 1 + 0 + 1 + 4^2} = \sqrt{18} \approx 4.243$.
- **Boa constrictor dart frog:** $d = \sqrt{1 + 1 + 1 + 1 + 4^2} = \sqrt{20} \approx 4.472$.

The result is reassuring: the two snakes are much closer to each other than either is to the dart frog, which matches the intuition that two snakes should look more alike than a snake and a frog. At this stage, the representation and the metric are working together correctly.

## The alligator puzzle: one feature hijacks the metric

Adding a fourth animal, the alligator $[1, 1, 0, 1, 4]$ — note the 4 in the legs position, just like the frog — and recomputing the matrix gives a surprise:

- Alligator rattlesnake: $d = \sqrt{0 + 0 + 1 + 0 + 16} = \sqrt{17} \approx 4.123$

- Alligator  boa constrictor: $d = \sqrt{1+0+0+0+16} = \sqrt{17} \approx 4.123$
- Alligator  dart frog: $d = \sqrt{0+1+1+1+0} = \sqrt{3} \approx 1.732$

The alligator is declared *closer to the dart frog* than to either snake. Yet counting features says otherwise: the alligator differs from the frog in **three** features but from the boa in only **two**, so by feature count it should be nearer the boa.

The culprit is the **scales of the features**. The legs dimension runs from 0 to 4, while every other feature runs from 0 to 1, so the legs axis is disproportionately large. The alligator and the frog share four legs while the snakes have none, and that gap of 4 — squared inside the distance formula — contributes 16 to the sum, swamping every binary disagreement, each of which contributes at most 1. Because squared differences are added, a single wide-range feature ends up dominating the calculation: the metric is effectively measuring "does it have legs?" rather than overall similarity. Nothing is wrong with Euclidean distance itself; the problem is that the features are not commensurable.

## Repairing the representation: binarize the offending feature

The fix changes *how the data is represented*, not how distance is computed. Instead of counting legs, the feature simply asks whether the animal has legs at all. The new encodings are:

- Rattlesnake: $[1, 1, 1, 1, 0]$, boa constrictor: $[0, 1, 0, 1, 0]$ (unchanged)
- Dart frog: $[1, 0, 1, 0, 1]$
- Alligator: $[1, 1, 0, 1, 1]$

Recomputing the matrix transforms the picture. The circled entries — rattlesnake  boa, rattlesnake  alligator, and boa  alligator — are all $\sqrt{2} \approx 1.414$: the three reptiles form an equilateral cluster. Distances to the dart frog are all larger: $\sqrt{3} \approx 1.732$ from the rattlesnake, $\sqrt{5} \approx 2.236$ from the boa constrictor, and $\sqrt{3} \approx 1.732$ from the alligator. Now the alligator sits with the snakes rather than the frog, which makes biological sense. The distance computation never changed — only the encoding of one feature did.

## The moral: feature engineering matters

The red-box takeaway from the exercise is **feature engineering matters**. The entire behavioral change came from re-representing the data; the algorithm was untouched. This mirrors the formal definition: in supervised machine learning and statistical modeling, feature engineering is a *preprocessing step* that transforms raw data into a more effective set of inputs, where each input comprises several attributes known as features. By providing models with relevant information, it significantly enhances predictive accuracy and decision-making capability.

The practice spans several activities: creating features from existing data, transforming and imputing missing or invalid features, reducing dimensionality through methods such as Principal Components Analysis (PCA), Independent Component Analysis (ICA), and Linear Discriminant Analysis (LDA), and selecting the most relevant features for training based on importance scores and correlation matrices. Several of its lessons surface directly in the alligator example:

- **Features vary in significance.** Even relatively insignificant features may contribute to a model, and feature selection — reducing the number of features — helps prevent a model from becoming too specific to the training data (overfitting).
- **Too many or badly scaled features cause trouble.** *Feature explosion* occurs when the identified features are too numerous for effective model estimation or optimization; it is limited via regularization, kernel methods, and feature selection.
- **It is hard work.** Feature engineering is time-consuming and error-prone, requiring domain expertise and trial and error — deciding that "leg count" should become "has legs" is exactly the kind of judgment call that demands knowing the domain. As an alternative, deep learning algorithms can process a large raw dataset without resorting to manual feature engineering.

The example also connects to clustering specifically: feature engineering has long been applied to clustering feature-objects or sample-objects in a dataset. Matrix-decomposition methods under non-negativity con-

straints — Non-Negative Matrix Factorization (NMF), Non-Negative Matrix-Tri Factorization (NMTF), Non-Negative Tensor Decomposition/Factorization (NTF/NTD) — produce part-based representations whose factor matrices exhibit natural clustering properties. The alligator story is the same principle in miniature: whichever representation you feed in determines which groupings come out.

## Two Flavors of Learning: Unsupervised versus Supervised

Every learning algorithm discussed in the coming lectures falls into one of two families, distinguished by whether the training data carries labels.

In **unsupervised learning**, we receive *unlabeled* data — nobody has told us what class any example belongs to. The goal is to discover structure from the data itself: we find clusters of examples that lie near each other, adopt the **centroids** of those clusters as the definitions of the learned classes, and then classify new data simply by assigning it to the closest cluster. No teacher ever provided answers; the classes emerge from the geometry of the data.

In **supervised learning**, we instead receive *labeled* data — example input-output pairs — and learn a mathematical surface that "best" separates the labeled examples, where the scare quotes around "best" matter because defining what makes a separation good is a deep question in itself. Crucially, this search is *subject to constraints on the complexity of the surface*, precisely because an unconstrained surface will overfit. When new data arrives, we assign it to a class based on which portion of the feature space, carved out by the classifier surface, it lies in.

The Wikipedia framing reinforces both pictures. Supervised learning trains a statistical model on labeled data, where the term "supervised" refers to a teacher providing correct outputs — e.g., many images explicitly labeled "cat" — and success is measured by **generalization error**: how accurately the model predicts outputs for new, unseen data. Its canonical tasks are classification (predicting a category, like spam vs. not spam) and regression (predicting a continuous value, like house prices). And there is no single algorithm that works best on all problems — the **no free lunch theorem** guarantees that every method has strengths and weaknesses, so the choice of approach always matters.

## Design Choices That Shape Any Learned Model

Whichever flavor of learning we pursue, the learned model depends on choices *we* make, not just on the data:

1. **The distance metric** — what does it even mean for two examples to be "near" each other?
2. **The feature vectors** — what do we choose to measure about each example in the first place?
3. **Constraints on model complexity** — a specified number of clusters, or a bound on the complexity of the separating surface.

Constraint (3) exists to prevent **overfitting**, the degenerate situation where each example becomes its own cluster, or where we draw an enormously complex separating surface that fits the training data perfectly but has learned nothing generalizable. The Wikipedia treatment sharpens why this is dangerous: overfitting is the production of an analysis that corresponds too closely to one particular dataset and therefore fails to predict future observations reliably — an overfitted model contains *more parameters than can be justified by the data*, unknowingly extracting residual variation (noise) as if it were underlying structure. In the extreme, if the number of parameters equals or exceeds the number of observations, the model can "predict" the training data perfectly by memorizing it in its entirety, and will typically fail severely on new data. The mirror-image failure is **underfitting**: a model missing terms needed to capture the data's structure, such as fitting a linear model to nonlinear data, which also yields poor predictions. Remedies include cross-validation, regularization, early stopping, pruning, Bayesian priors, and dropout — techniques that either penalize overly complex models explicitly or test generalization on held-out data — guided by the **principle of parsimony**: balance the errors of underfitting and overfitting rather than eliminating either alone.

The supervised-learning literature organizes these concerns into four major issues that map directly onto the lecture's list. First, the **bias–variance tradeoff**: an algorithm is biased if it is systematically incorrect for an input across different training sets, and has high variance if it predicts different values when trained

on different sets; prediction error relates to their sum. Low bias demands a flexible model, but too much flexibility means the model fits each training set differently — high variance — so methods expose a knob to adjust this tradeoff. Second, the **amount of training data relative to the complexity of the true function**: simple functions can be learned from little data by inflexible high-bias algorithms, while complex functions require large datasets paired with flexible low-bias ones. Third, the **dimensionality of the input space**: extra irrelevant dimensions confuse the learner and inflate variance, so manually removing irrelevant features, automated feature selection, or dimensionality reduction improves accuracy. Fourth, the **degree of noise in the target values**: if outputs are often wrong (human or sensor error), the algorithm should not try to match them exactly — doing so causes overfitting. Notably, you can overfit even with clean measurements if the true function is too complex for your model, a phenomenon called *deterministic noise*; in either case, a higher-bias, lower-variance estimator is preferable.

## Clustering as Iterative Refinement

Suppose we know there are $k$ distinct groups in our training data but do not know the labels. The natural recipe given in lecture is:

1. Pick $k$ samples — at random, perhaps — to serve as **exemplars**.
2. Cluster the remaining samples by minimizing the distance between samples in the same cluster; this minimization criterion is the **objective function**. In practice, this means assigning each sample to the group whose exemplar is closest.
3. Find the median example in each cluster and promote it to be the new exemplar.
4. Repeat until there is no change.

This is exactly the family of algorithms known as **$k$-means clustering**, a vector-quantization method originating in signal processing that partitions $n$ observations into $k$ clusters, each observation belonging to the cluster with the nearest mean (centroid). Formally, given observations $(x_1, x_2, \dots, x_n)$, the objective is to minimize the **within-cluster sum of squares (WCSS)**:

$$\arg\min_{\mathbf{S}} \sum_{i=1}^{k} \sum_{\mathbf{x} \in S_i} \left\| \mathbf{x} - \mu_i \right\|^2 \;=\; \arg\min_{\mathbf{S}} \sum_{i=1}^{k} |S_i| \operatorname{Var}(S_i)$$

where the centroid of cluster $S_i$ is

$$\mu_i = \frac{1}{|S_i|} \sum_{\mathbf{x} \in S_i} \mathbf{x},$$

and $\| \cdot \|$ is the usual $L^2$ norm. This is equivalent to minimizing the pairwise squared deviations of points within the same cluster, via the identity

$$|S_i| \sum_{\mathbf{x} \in S_i} \left\| \mathbf{x} - \mu_i \right\|^2 = \frac{1}{2} \sum_{\mathbf{x}, \mathbf{y} \in S_i} \left\| \mathbf{x} - \mathbf{y} \right\|^2,$$

and, since total variance is constant, to maximizing the between-cluster sum of squares — a relationship tied to the law of total variance. The resulting partition carves the data space into **Voronoi cells** around the centroids.

Two subtleties deserve attention. First, note the mismatch between the lecture's "median" update step and the classical mean: $k$-means minimizes *squared* Euclidean distances, and the mean is what optimizes squared errors — minimizing plain Euclidean distances would be the harder Weber problem, requiring the geometric median, which is why variants like $k$-medians and $k$-medoids exist for better Euclidean solutions. Second, the exact problem is computationally difficult (**NP-hard**), yet efficient heuristic algorithms converge quickly to a local optimum. The standard heuristic, **Lloyd's algorithm** — proposed by Stuart Lloyd at Bell Labs

in 1957 for pulse-code modulation and independently published by Edward Forgy in 1965, hence "Lloyd–Forgy" — alternates between an assignment step (each point goes to its nearest current mean) and an update step (recompute the means), exactly mirroring the lecture's loop of assign-then-recompute-exemplars. After each iteration the WCSS monotonically decreases, giving a nonnegative monotonically decreasing sequence and a convergence guarantee. The approach is closely related to expectation–maximization for Gaussian mixture models — both use cluster centers to refine iteratively — though $k$-means tends to find clusters of comparable spatial extent while Gaussian mixtures allow different shapes. Known limitations motivated extensions: sensitivity to initial centroid placement, difficulty with non-spherical clusters, fuzzy $c$-means (partial membership in multiple clusters), and kernel $k$-means (non-linearly separable clusters).

Finally, the classification rule for new data has a precise name: applying the 1-nearest-neighbor classifier to the cluster centers obtained by $k$-means classifies new points into existing clusters — the **nearest centroid classifier**, also known as the Rocchio algorithm. This is the formal version of "assign the new dot to the closest cluster," and despite the similar names, it should not be confused with the supervised $k$-nearest-neighbor classifier.

## A Worked Example: Height versus Weight, and Why $k$ Changes Everything

To make the machinery concrete, consider a distribution of height versus weight: height runs along the $x$-axis from 60 to 90, weight along the $y$-axis from roughly 200 to 350, and each dot is one example. In the unsupervised setting, *all we see are the dots* — we have no idea which animal, so to speak, each dot came from.

**Fitting two clusters.** Asking the algorithm for $k = 2$ yields one group circled in red dashed lines — points at heights of roughly 72 to 80 and weights of roughly 265 up to about 335 — and another circled in green dashed lines down at heights of roughly 68 to 74 and weights of roughly 190 to 210. But notice a black point sitting right near the top of the lower circle, at about height 72 and weight 250: it lies right on the boundary, and it is genuinely ambiguous which cluster it belongs to. This is the practical face of the Voronoi-cell picture — points near a cell boundary have nearly equidistant centroids, so the assignment is fragile.

**Fitting three clusters.** Now fit $k = 3$ on the *same data, nothing else changed*. The red dashed circle shrinks to capture just the topmost points; a green dashed circle appears in the middle, capturing the two points at weights around 265 to 275 together with that formerly awkward black point at 250; and a blue dashed circle at the bottom captures the remaining points down around weights 190 to 210. The ambiguous point got absorbed into a newly created middle cluster.

**The lesson.** Same unlabeled data, a different choice of $k$, and we get a completely different "learning." Which is right? The data cannot tell us — there are no labels to check against. This is precisely the issue flagged earlier: how do we decide on the best number of clusters, and how do we choose the best features and the best distance metric? Those choices drive the answer, and managing them — without overfitting, without letting every example become its own cluster — is the real art of unsupervised learning.

## Overfitting: Better on Training Data, Worse Where It Counts

Recall that our two simple voter-classification models — the solid-line and dashed-line boundaries — both scored an accuracy of 0.7 on the training data, where accuracy is the fraction of all examples classified correctly:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Since neither simple model wins, the natural temptation is to ask: can we just do better on the training set? We can — by adding complexity. A far more elaborate boundary, a wiggly path snaking around the plot, contorts itself to separate the blue stars (Democrats) from the red triangles (Republicans). Evaluated on the training data it achieves $TP = 12$, $FP = 5$, $TN = 13$, and remarkably $FN = 0$, giving an accuracy of $25/30 \approx 0.833$ — clearly better than 0.7.

But when the *same* boundary is applied to test data — points the model has never seen — the picture collapses: $TP = 14$, $FP = 4$, $TN = 4$, $FN = 8$, for an accuracy of only $18/30 = 0.6$. That is worse than what either simple model achieved on its training data. The complex model was so busy fitting the idiosyncrasies of the training set that it captured nothing generalizable.

This phenomenon has a name: **overfitting**. In mathematical modeling, overfitting is the production of an analysis that corresponds too closely or exactly to a particular set of data, and therefore fails to fit additional data or predict future observations reliably. An overfitted model contains more parameters than can be justified by the data, and its essence is unknowingly extracting some of the residual variation — the noise — as if that variation represented the underlying structure. In machine-learning terms, the model begins to *memorize* the training data rather than *learn* to generalize from a trend. The extreme case makes the failure vivid: if the number of parameters equals or exceeds the number of observations, a model can perfectly predict the training data simply by memorizing it in its entirety — and will typically fail severely when making predictions. In regression, the same trap appears whenever there are $p$ variables and $p$ data points: the fitted line passes exactly through every point while predicting nothing.

Two further points from the theory sharpen the lesson. First, even a model without excessive parameters will usually appear to perform somewhat worse on new data than on the data used for fitting — a phenomenon called *shrinkage* — so some gap between training and test performance is expected. Second, overfitting has a mirror image, **underfitting**: a model missing parameters or terms needed to capture the data's structure, such as fitting a linear model to nonlinear data, which also yields poor predictive performance. Good modeling balances the two error sources — the bias–variance tradeoff — and adheres to the Principle of Parsimony (Occam's razor): prefer no more adjustable parameters or complexity than is ultimately optimal.

How do we guard against overfitting? The standard techniques fall into two families: those that explicitly penalize overly complex models (regularization, pruning, Bayesian priors, early stopping, dropout), and those that test generalization directly by evaluating performance on data not used for training — cross-validation being the canonical example. Our lecture's train/test split is exactly this second strategy in action, and it is why performance on test data, not training data, is what we actually care about.

## Why Accuracy Alone Can Mislead: PPV, Sensitivity, and Specificity

The overfitting episode also shows that accuracy may not be the right way to compare models, because different metrics can tell different stories. **Positive predictive value** asks: of all the examples we labeled positive, how many really were positive?

$$\text{PPV} = \frac{TP}{TP + FP}.$$

For our models: the solid line scores 0.57, the dashed line 0.58, the complex model 0.71 on training data — and, strikingly, 0.78 on *test* data. By this measure the complex model actually looks better than the simple ones even out of sample, the opposite of what accuracy suggested. Which metric matters depends on your application.

Two more terms you will encounter constantly in the literature, especially in medical testing contexts:

- **Sensitivity** — the percentage of positives correctly found:

$$\text{Sensitivity} = \frac{TP}{TP + FN}.$$

- **Specificity** — the percentage of negatives correctly rejected:

$$\text{Specificity} = \frac{TN}{TN + FP}.$$

Note how the complex model's zero false negatives on training data would give it perfect sensitivity there, yet its eight false negatives on test data drag that same metric down — another illustration that a single number, computed on a single dataset, rarely tells the whole story.

## The Two Paradigms of Machine Learning — and What Shapes Their Results

Stepping back, machine learning provides a way of building models of processes from datasets, and it comes in two main flavors:

- **Supervised learning** uses *labeled* data to create classifiers that optimally separate examples into known classes — exactly what we did with our voters, where each training point came tagged Democrat or Republican.
- **Unsupervised learning** tries to infer latent variables by clustering training examples into nearby groups — here nobody tells you the labels; the structure must emerge from the data itself.

Across every experiment in this lecture, two design decisions repeatedly proved decisive: the **choice of features** influences the results, and the **choice of distance measurement** between examples influences the results. These choices matter just as much as the algorithm itself.

Looking ahead, the coming lectures develop concrete instances of both paradigms: clustering methods such as **k-means** on the unsupervised side, and classifier methods such as **k nearest neighbors** on the supervised side.